Advance Tokenization methods

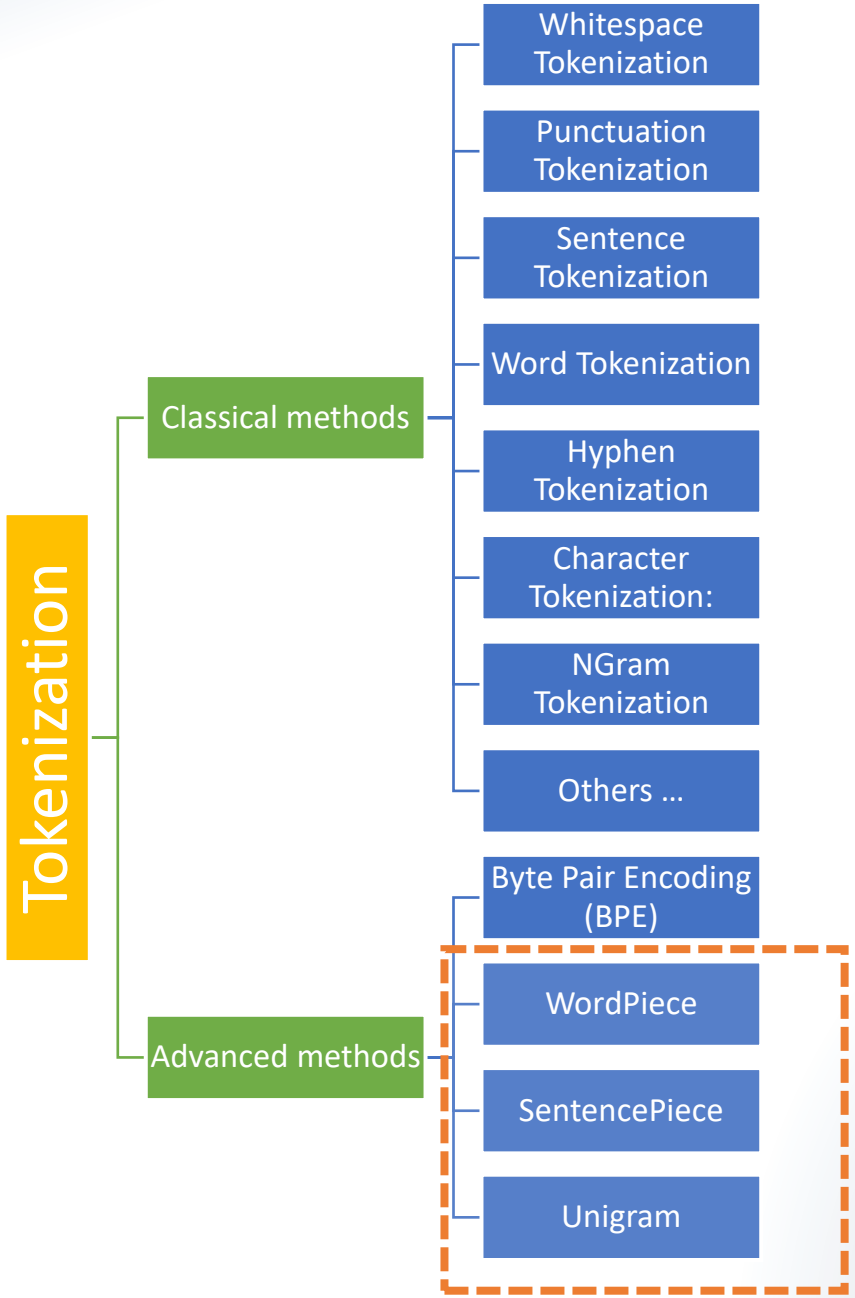
Textual Alchemy:

The Art and Science of Tokenization.

3 main types of tokenizers used in Transformers

- Byte Pair Encoding (BPE)
- WordPiece
- SentencePiece (Unigram or BPE)

Topics


Tokenization
Classical methods
Whitespace Tokenization
Punctuation Tokenization
Sentence Tokenization
Word Tokenization
Hyphen Tokenization
Character Tokenization:
NGram Tokenization
Others …
Advanced methods
Byte Pair Encoding (BPE)
WordPiece
SentencePiece
Unigram

Tokenization

breaking down a text into smaller units

Why tokenize?

A fundamental reason we need to tokenize text in AI and natural language processing (NLP) is to convert unstructured textual data into a structured format that can be effectively understood, processed, and analyzed by machines.

"The quick brown fox jumped over the lazy dog."

["The", "quick", "brown", "fox", "jumped", "over", "the", "lazy", "dog", "."]

“dog" = [0, 0, 1, 0, 0, ...]; “brown" = [0, 1, 1, .9, 0, ...]

Tokenization – key points

01

a fundamental text preprocessing technique

02

primary purpose is to break down a piece of text, typically a sentence or document, into smaller units called tokens.

03

Tokens are the individual elements or building blocks of text, which are usually words or subword units.

Byte-Pair Encoding (BPE)

Byte-Pair Encoding (BPE) was initially developed as an algorithm to compress texts, and then used by OpenAI for tokenization when pretraining the GPT model.

used by a lot of Transformer models, including GPT, GPT-2, RoBERTa, BART, and DeBERTa.

methods for tokenization in transformers

Byte-Pair Encoding (BPE)

- Divides words into subword units based on frequency of co-occurrence of byte pairs.
- Implemented in tokenizers like BertTokenizer and OpenAIGPTTokenizer.

WordPiece

- Similar to BPE but segments words into smaller units based on a language model's vocabulary.
- Used in tokenizers such as DistilBertTokenizer and RoBERTaTokenizer.

SentencePiece

- Provides an unsupervised text tokenization method that can tokenize text into subwords or even whole words.
- Applied in tokenizers like BartTokenizer and MBartTokenizer.

methods for tokenization in transformers

Word-Level Tokenization	Character-Level Tokenization	Byte-Level Byte-Pair Encoding (BBPE)
• Tokenizes text at the word level, where each word is treated as a single token. • Seen in tokenizers such as AlbertTokenizer and XLMRobertaTokenizer.	• Breaks down text into individual characters, treating each character as a separate token. • Utilized in tokenizers like ConvBertTokenizer.	• Extends BPE to operate at the byte level, which can be beneficial for handling multilingual text and special characters. • Implemented in tokenizers like GPT2Tokenizer.

methods for tokenization in transformers

Tokenizer for Sequence Classification

- Tokenizes sequences specifically for tasks like sentiment analysis or text classification.
- Examples include ElectraTokenizer.

Tokenizer for Named Entity Recognition (NER)

- Specialized tokenization for identifying and classifying entities within text.
- Seen in tokenizers like XLMTokenizer.

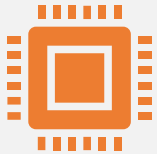
Fast Tokenization

- Optimized tokenization methods designed for faster processing, often with reduced vocabulary or streamlined algorithms.
- Examples include DistilBertTokenizerFast and RobertaTokenizerFast.

Tokenization for Specific Applications

- Tailored tokenization methods designed for specific tasks or domains, such as social media text (e.g., BertweetTokenizer) or multilingual text processing.

Byte Pair Encoding (BPE)



Byte Pair Encoding (BPE) is a data compression technique that has found applications beyond compression

Specially, in the field of natural language processing (NLP) for sub word tokenization.

BPE is used to progressively merge the most frequent pairs of characters or sub word units in a given corpus until a **desired** vocabulary size is reached.

Illustration of how Byte Pair Encoding works

let’s say our corpus uses these five words: "hug", "pug", "pun", "bun", "hugs“

base vocabulary (sorted) will then be ["b", "g", "h", "n", "p", "s", "u"].

For real-world cases, that base vocabulary will contain all the ASCII characters, at the very least, and probably some Unicode characters as well.

If an example you are tokenizing uses a character that is not in the training corpus, that character will be converted to the unknown token.

Frequencies

let’s assume the words had the following frequencies:

- ("hug", 10),
- ("pug", 5),
- ("pun", 12),
- ("bun", 4),
- ("hugs", 5)

meaning

- "hug" was present 10 times in the corpus,
- "pug" 5 times,
- "pun" 12 times,
- "bun" 4 times, and
- "hugs" 5 times.

Training the tokenizer

- splitting each word into characters (the ones that form our initial vocabulary) so we can see each word as a list of tokens:
  - ("h" "u" "g", 10),
  - ("p" "u" "g", 5),
  - ("p" "u" "n", 12),
  - ("b" "u" "n", 4),
  - ("h" "u" "g" "s", 5)

- pair ("h", "u") is present in the words "hug" and "hugs", so 15 times total in the corpus.
- ("u", "g"), which is present in "hug", "pug", and "hugs", for a grand total of 20 times in the vocabulary.
- Merging Most Frequent Pair: Identify the most frequent pair of consecutive characters (or subwords) in the current dataset. Merge this pair into a single token and add it to the vocabulary.

Merge

- the first merge rule learned by the tokenizer is ("u", "g") -> "ug", which means that "ug" will be added to the vocabulary, and the pair should be merged in all the words of the corpus.

- At the end of this stage, the vocabulary and corpus look like this:

  - Vocabulary: ["b", "g", "h", "n", "p", "s", "u", "ug"]

  - Corpus: ("h" "ug", 10), ("p" "ug", 5), ("p" "u" "n", 12), ("b" "u" "n", 4), ("h" "ug" "s", 5)

Merge

- the pair ("h", "ug"), for instance (present 15 times in the corpus).
- most frequent pair at this stage is ("u", "n"), however, present 16 times in the corpus,
-
- So the second merge rule learned is ("u", "n") -> "un".
- Adding that to the vocabulary and merging all existing occurrences leads us to:
  - Vocabulary: ["b", "g", "h", "n", "p", "s", "u", "ug", "un"]
  - Corpus: ("h" "ug", 10), ("p" "ug", 5), ("p" "un", 12), ("b" "un", 4), ("h" "ug" "s", 5)

… next

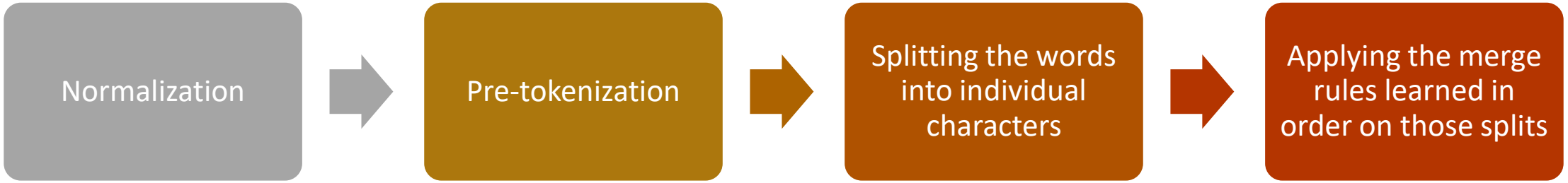
- Now the most frequent pair is ("h", "ug")
- So, we learn the merge rule ("h", "ug") -> "hug"
- which gives us our first three-letter token.
- After the merge, the corpus looks like this:

Vocabulary: ["b", "g", "h", "n", "p", "s", "u", "ug", "un", "hug"]

Corpus: ("hug", 10), ("p" "ug", 5), ("p" "un", 12), ("b" "un", 4), ("hug" "s", 5)

we continue like this until we reach the desired vocabulary size.

BPE Tokenization algorithm



Let's take the example we used during training, with the three merge rules learned:
("u", "g") -> "ug"
("u", "n") -> "un"
("h", "ug") -> "hug"

The word "bug" will be tokenized as ["b", "ug"]. "mug", however, will be tokenized as ["[UNK]", "ug"] since the letter "m" was not in the base vocabulary.

Likewise, the word "thug" will be tokenized as ["[UNK]", "hug"]: the letter "t" is not in the base vocabulary

transformer models which use BPE

BERT (Bidirectional Encoder Representations from Transformers):

- BERT, developed by Google, uses WordPiece tokenization, which is a variant of BPE.

GPT (Generative Pre-trained Transformer) Series:

- The GPT series, including models like GPT-2 and GPT-3, uses BPE for subword tokenization.

RoBERTa (Robustly optimized BERT approach):

- RoBERTa, developed by Facebook AI, uses Byte Pair Encoding (BPE) for subword tokenization.

XLM-R (Cross-lingual Language Model - Revised):

- XLM-R, a cross-lingual transformer-based language model, utilizes BPE for subword tokenization.

transformer models which use BPE

T5 (Text-to-Text Transfer Transformer)

- T5, developed by Google, uses SentencePiece, which is another variant of BPE, for subword tokenization.

ALBERT

- ALBERT, an optimized version of BERT, also uses a variant of BPE for subword tokenization.

DistilBERT (Distill BERT):

- DistilBERT, a distilled version of BERT, uses a variant of BPE for subword tokenization.

ERNIE (Enhanced Representation through kNowledge Integration):

- ERNIE, developed by Baidu, uses a variant of BPE for subword tokenization.

Advantages of BPE

Flexibility in Vocabulary Handling:

Because of subword units, allows the model to effectively handle out-of-vocabulary terms and morphological variations.

Adaptability to Multiple Languages:

BPE is language-agnostic

Efficient Handling of Rare Words:

BPE excels in capturing subword units, which makes it effective in representing and processing rare words or words not present in the training vocabulary.

Improves Generalization:

BPE aids in generalizing

Limitations of BPE

Computational Complexity

- training of BPE involves iteratively merging the most frequent pairs of subword units, which can be computationally expensive and time-consuming, especially for large datasets.

Vocabulary Size

- BPE may lead to a large vocabulary size, especially when considering subword units.

Loss of Interpretability

- Subword tokenization may result in tokens that do not directly correspond to meaningful linguistic units, leading to a potential loss of interpretability compared to word-level tokenization.

Demo using python/Hugging face

(Implement BPE)

Thanks